NAME	**SHARMILA S**
REGISTER NUMBER	**1912260141**
DEPARTMENT	**B.TECH-ECE**
COURSE NAME	**PYTHON PROGRAMMING**
COURSE CODE	**CSA0823**
SLOT	**C**
GITHUB LINK	**https://github.com/ssharmila8125-netizen/sharmisCSA08-1912260141/blob/main/assignment%202.pdf**

Q1. Decisions with Many Boundaries

Prediction table

UNITS	RATION CARD	AGE ABOVE 80	RESULT
50	NO	NO	FULL SUBSIDY
100	NO	NO	FULL SUBSIDY
101	YES	NO	PARTIAL SUBSIDY
150	NO	NO	NO SUBSIDY
200	YES	NO	PARTIAL SUBSIDY
201	YES	NO	NO SUBSIDY
250	NO	YES	SPECIAL SUBSIDY
250	NO	NO	NO SUBSIDY

2. Why separate if statements can give multiple results

Separate if statements are checked independently. More than one condition may be true, so multiple decisions may be printed. An if-elif-else structure checks conditions in order and executes only one matching branch.

3. Program using separate if statements

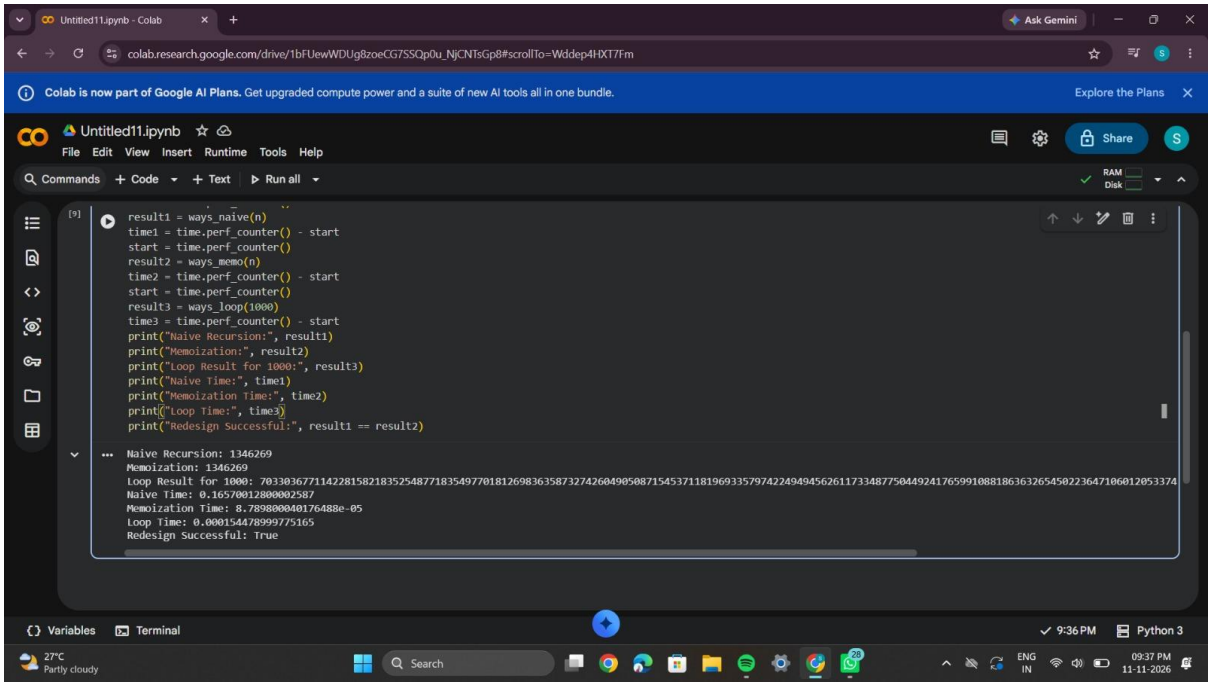
4. Program using if-elif-else

5. Design decision

Combined Boolean conditions with if-elif-else are easier to read and maintain. They clearly represent the eligibility rules and prevent multiple decisions.

6. Alternative concept – match-case

A match-case approach can also be used, but the conditions need to be arranged carefully. A lookup list searched in a loop takes O(k)time for k slabs. An if-elif-else chain also takes up to O(k) time. Both use O(1) extra space for a fixed number of conditions; a lookup list requires O(k) storage.



Q2. Loops that Count Attempts (4 Marks)

1. Using a while loop

2. Using a for loop

3. Uses of loop control statements

- break: Stops the loop when the correct OTP is entered.
- continue: Skips the current iteration for a blank entry.
- pass: Does nothing; it can be used as a placeholder in an unfinished block.
- else: Executes when the loop finishes without break.

4. While loop vs for loop

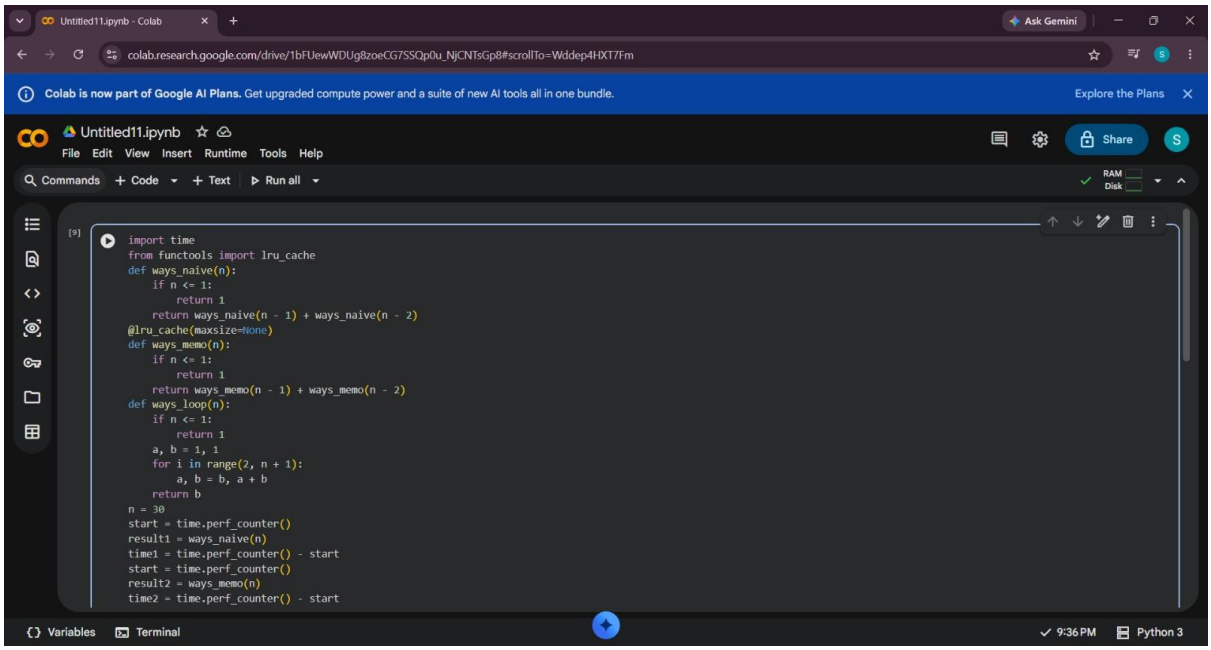
- A `while` loop is natural when the number of attempts depends on a condition. A `for` loop is suitable when the number of attempts is fixed.
- If the counter is updated in the wrong place, the program may allow extra attempts or count blank entries as attempts.
- 5. Changed condition – two extra attempts
- The customer gets five attempts if the security question is answered correctly.

6. Alternative concept – recursion

Complexity analysis

METHOD	TIME COMPLEXITY	EXTRA SPACE
while loop	O(a)	O(a)
for loop	O(a)	O(a)
recursion	O(a)	O(a)

Here, a is the maximum number of attempts. Recursion uses extra stack space, so loops are more suitable for OTP verification.



Q3. Vehicle Registration Number Validator (6 Marks)

1. Normalisation

The program removes spaces and hyphens and converts the registration number to uppercase.

2. Complete Python program

3. Six predicted test inputs

INPUT	EXPECTED RESULTS
TN 09 AB 1234	Valid
KA 01 M 0007	Valid
MH 12 XYZ 1234	Invalid
TN9AB1234	Invalid
KL 05 C 6789	Valid
TN 22 1234 AB	Invalid

4. Why strings are immutable

Strings are immutable because their characters cannot be changed after creation. Any modification creates a new string.

text.upper() returns a new uppercase string. It does not change the original string.

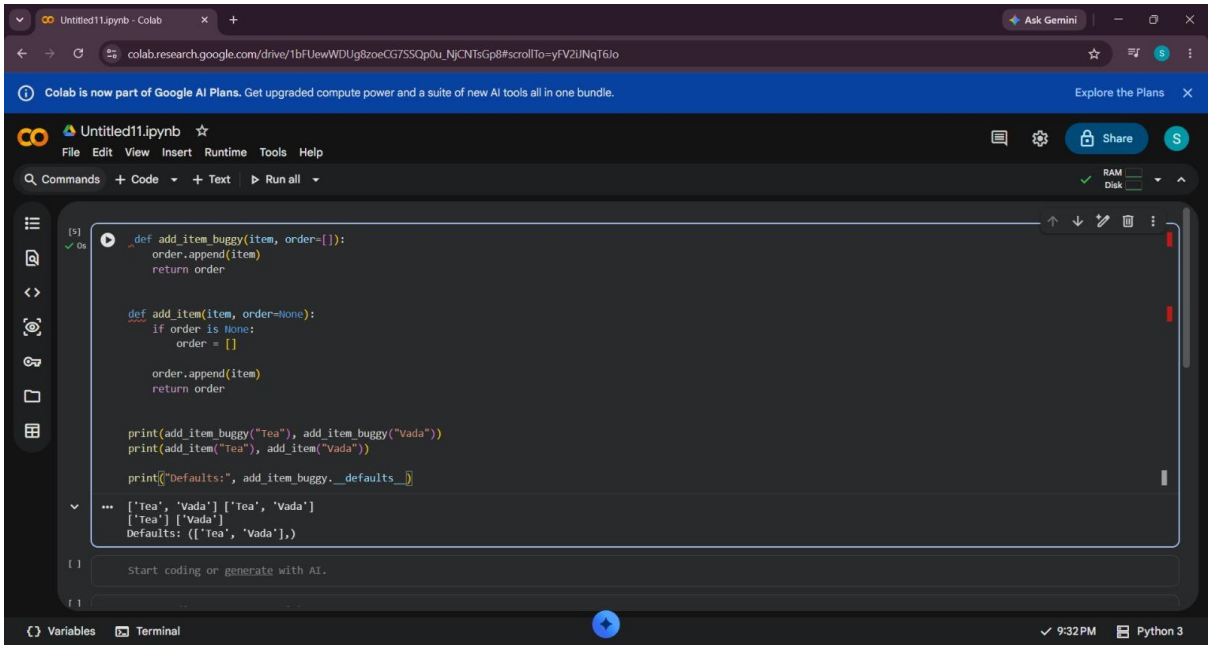
5. Slicing explanation

- s[-4:] returns the last four characters.
- s[4:-4] returns the characters between the first four and last four characters.

For a 9-character string, s[4:-4] returns one character. For a 10-character string, it returns two characters.

6. Alternative concept – regular expression

Both slicing and regular-expression validation take \(O(L)\) time, where \(L\) is the string length. Regular expressions are compact, while slicing makes each validation step easy to understand and modify.



Q4. Building Long Text: + or join()? (6 Marks)

1. Three methods

- Repeated += adds each line to the existing string.
- join() combines a list of strings in one operation.
- An f-string formats each individual line before combining it.

2. Complete the given program

3. Third method – f-string

4. Why is join() recommended?

Strings are immutable. Repeated += may create new strings while building a large statement. join() combines the parts efficiently and is generally preferred for large collections of strings.

5. Alternative concept – io.StringIO

from io import StringIO

6. Complexity analysis

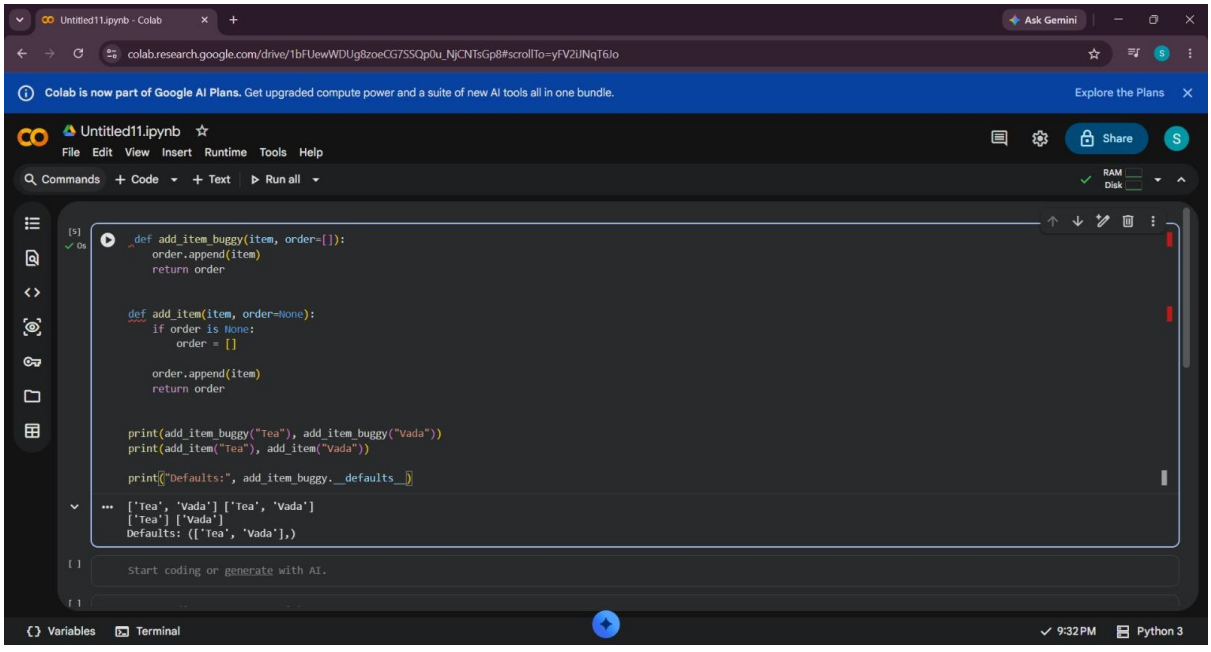
METHOD	TIME COMPLEXITY	EXTRA SPACE
Repeated	UP to O(n**2)	O(n) final string
Join()	O(n)	O(n)
stringIO	O(n)	O(n)

Here, \n represents the total number of characters. join() is suitable when all parts are already available. StringIO is useful when text is generated gradually. Writing directly to a file can reduce the need to keep the entire statement in memory.

7. Measurement evidence

Run the program three times for 1,000, 10,000 and 200,000 lines. Record the actual times and calculate:

Average=T1+T2+T3/3



Q5. Functions, Parameters and Scope (7 Marks)

1. Explanation of the default parameter bug

In add_item_buggy(item, order=[]), the empty list is created only once when the function is defined. Therefore, the same list is reused in every function call.

2. Corrected program

3. Types of parameters

- Positional parameters: Values are passed according to their position.
- Keyword parameters: Values are passed using parameter names.
- Default parameters: Values are automatically used when no argument is provided.

4. Returning more than one value

5. Local, global and nonlocal variables

- Local variable: Declared inside a function.
- Global variable: Declared outside a function.
- Nonlocal variable: Refers to a variable in the enclosing function.

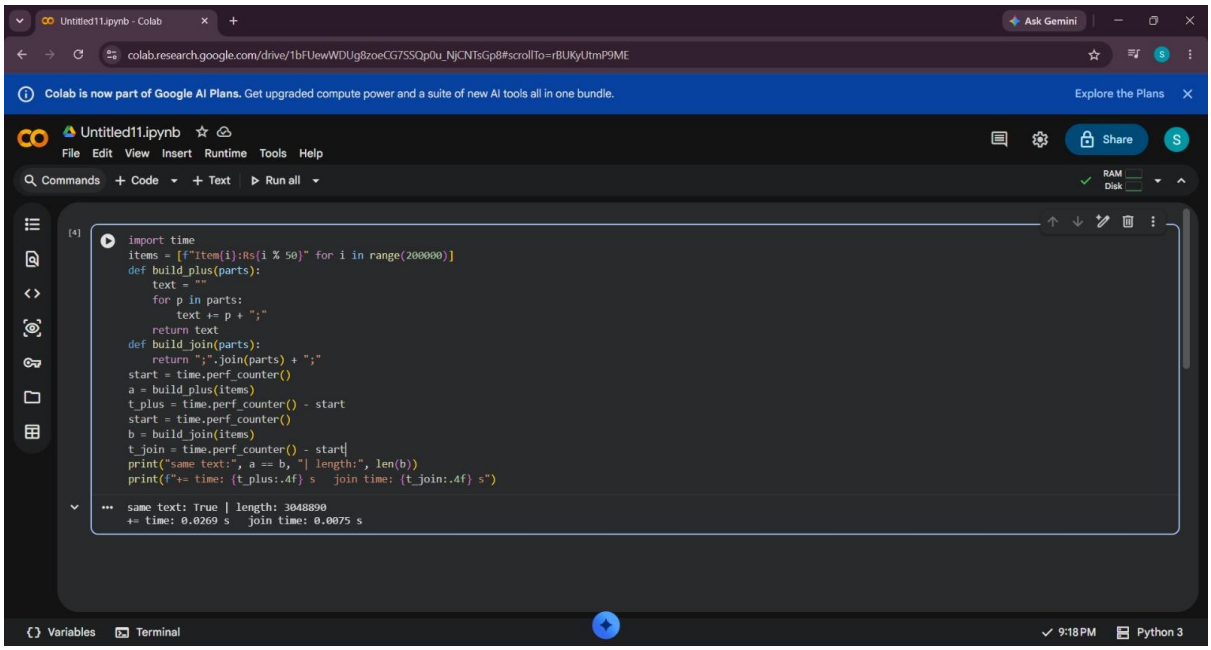
6. Why is the global keyword needed?

The global keyword allows a function to modify a global variable. It is usually avoided because it makes programs harder to understand and maintain.

7. Alternative concept

The caller can create and pass a separate list for every customer.

Each item insertion takes O(1) average time. The list requires O(n) space for \n items. Passing a separate list explicitly avoids sharing data between customers.



Q6. Recursion and Its Limits (8 Marks)

1. Base case and recursive case

- Base case: If n <= 1, return 1.
- Recursive case: Add the ways of climbing n-1 and n-2 steps.

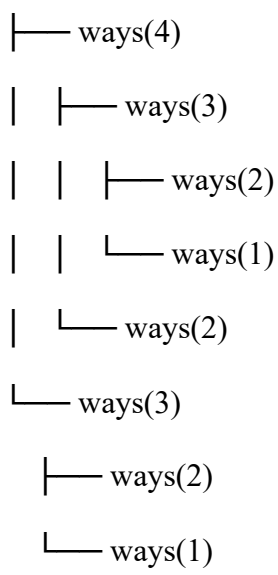
2. Complete memoisation program

3. Answers for n = 1 to 6

STEPS	NUMBER OF WAYS
1	1
2	2
3	3
4	5
5	8
6	13

4. Call tree for n = 5

ways(5)



The plain recursive method repeats calculations. Memoisation stores previously calculated answers and avoids repeating the same work.

5. Function call comparison

n	Native calls	Memorised calls
10	177	11
20	21891	21

25	242785	26
30	2692537	31

6. Recursion limit

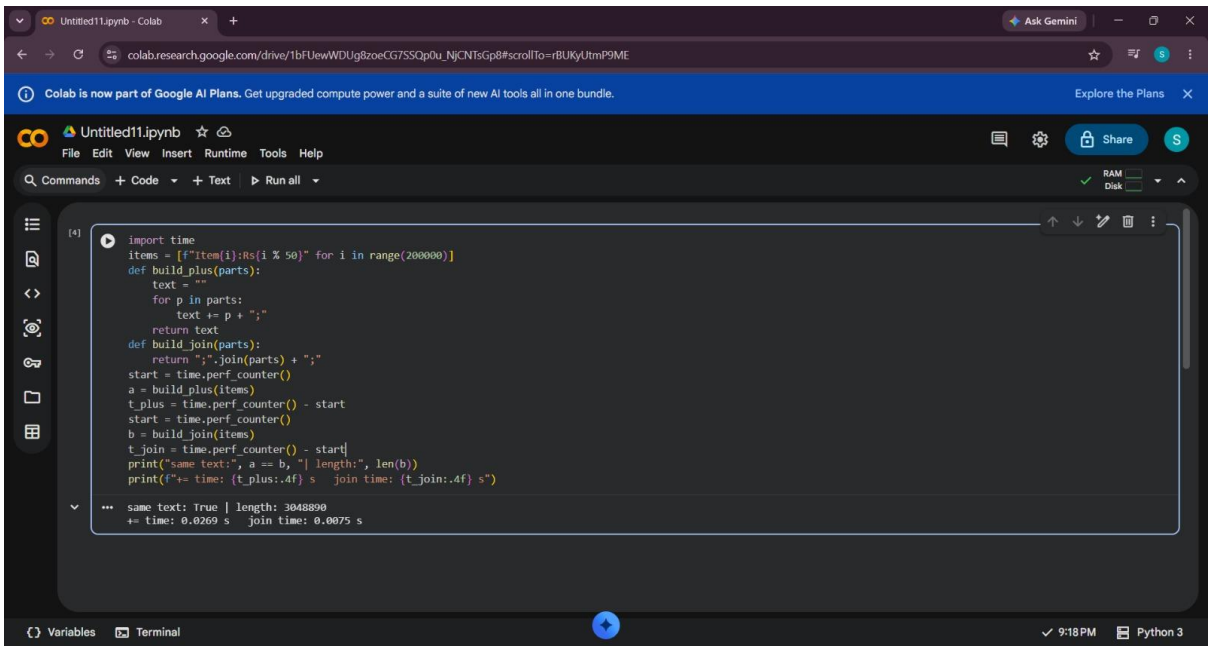
Python has a recursion limit to prevent excessive function calls. Calling ways_memo(1000) may produce a RecursionError because the recursive calls exceed the available recursion depth.

7. Alternative concept – iterative method

8. Complexity analysis

Method	**Time complexity**	**Space complexity**
Plain recursion	$O(2^n)$	O(n)
Memoisation	O(n)	O(n)
Iterative loop	O(n)	O(1)

The iterative method is the best choice for 1,000 steps because it avoids recursion errors and uses constant extra space.



Q7. Diagnose, Redesign and Validate (13 Marks)

Suggested answer using Q4 (string building) as the selected program.

A. Diagnose the bottleneck (2 Marks)

The original program uses repeated string concatenation with +=. Since strings are immutable, repeated concatenation can involve unnecessary copying of characters. The program becomes slower as the number of lines increases.

B. Define the design objective (2 Marks)

The objective is to reduce statement-generation time while producing exactly the same output.

C. Design the revised solution (4 Marks)

The revised program uses a list to store the statement parts and join() to combine them.

Flow:

Start

|

Read input lines

|

Store each line in a list

|

Use join() to combine lines

|

Generate final statement

|

Display output

|

Stop

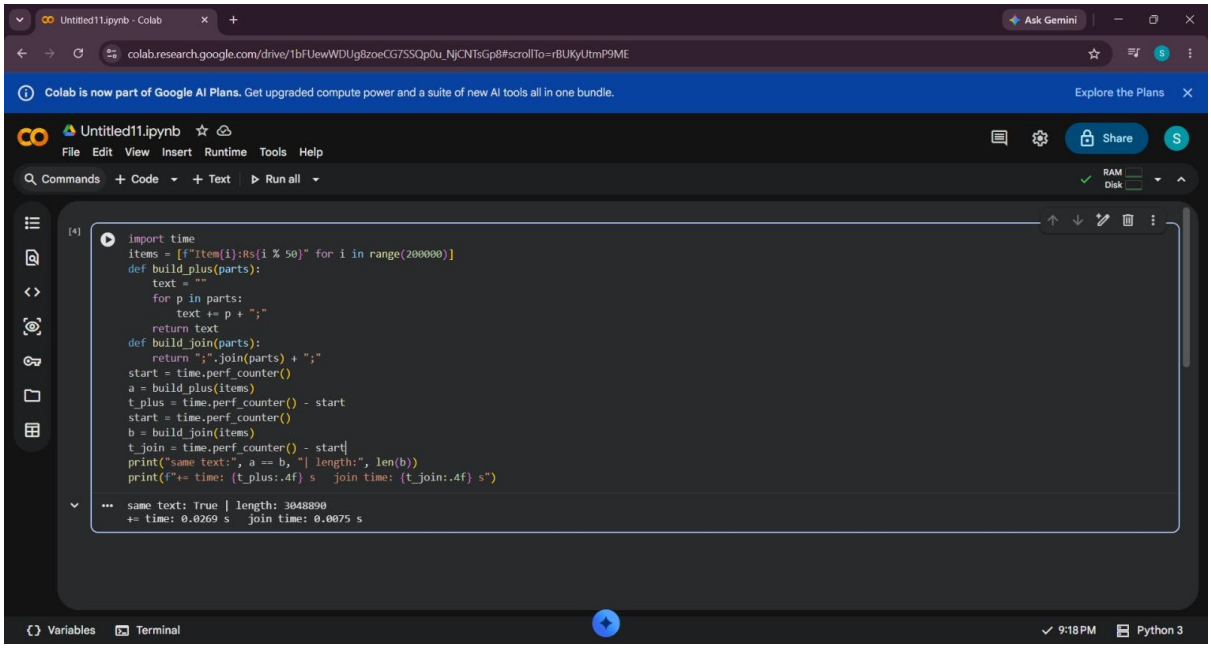
D. Predict before implementing (1 Mark)

I predict that the join() method will generate the statement faster than repeated +=, especially for large inputs.

E. Implement and validate

F. Defend the final design (2 Marks)

The redesigned solution produces the same output and is generally more efficient for large collections of strings. It may require extra memory to store the list of parts. The original method can still be suitable for very small strings.



Conclusion: The redesigned solution improves statement generation by reducing repeated string concatenation.